Module 4 Tangent:

Pattern Matching

And, of course, destructuring!

1.

Pattern Matching

I don’t want to create, I want to destroy!

Pattern Matching

Definition

Pattern matching allows testing values for conditions, often based on structure.

- This makes it easier to separate and address conditions one-by-one.
  - Also helps avoid overlooking one!
- Many modern languages are adding pattern matching - even Java!
  - `case Ast.Source source`

Destructuring

Destructuring is a key feature of pattern matching that extracts nested data.

- Lists (sequences) are most common:
  - `func map([], _)`
  - `func map([head, tail*], f)`
- Can also include maps/records (named destructuring) or even object types.
- A key part of destructuring involves **binding** extracted values to variables.

```java
Ast.Stmt.Assignment ast;
switch (ast.receiver()) {
  case Ast.Expr.Variable variable: {
    var name = variable.name();
    //implement variable assignment…
  }
  case Ast.Expr.Property property: {
    var name = property.name();
    var receiver = property.receiver();
    //implement property assignment…
  }
  default: {
    throw new EvaluateException("Receiver must be variable/property.");
  }
}
```

Java Pattern Matching

```
Ast.Stmt.Assignment ast;
switch (ast.receiver()) {
  case Ast.Expr.Variable(String name): {
    //var name = variable.name();
    //implement variable assignment…
  }
  case Ast.Expr.Property(var name, var receiver): {
    //var name = property.name();
    //var receiver = property.receiver();
    //implement property assignment…
  }
  default: {
    throw new EvaluateException("Receiver must be variable/property.");
  }
}
```

Java Pattern Matching & Destructuring

Why is pattern matching important for FP?

1. Combining conditions with data extraction is safer.
2. Immutablity means nested data can’t be modified!

```java
Optional<RuntimeValue> optional;

if (optional.isPresent()) {
  //access with .get() is safe(ish?)
  var value = optional.get();
}

//what if the code changes?
if (optional.isPresent() || true) {
  //access with .get() errors!
  var value = optional.get();
}

optional.map(value -> {
  //can't modify vars or throw >:(
});
```

```haskell
maybe :: Maybe RuntimeValue

case maybe of
  Just value → --value is safe!
  Nothing → --value isn't defined!

--in fact, this is the only way to
--access the value of a Maybe!
--(helper functions exist though)
```

Type Safety in Pattern Matching

Abusing Python

And Kotlin - they were BOGO!

Module 4 Tangent:

Pattern Matching

And, of course, destructuring!

Slides Carnival

Free templates for all your presentation needs

For PowerPoint and Google Slides

100% free for personal or commercial use

Ready to use, professional and customizable

Blow your audience away with attractive visuals